

# Markdown analyzer

**Aplikace pro analýzu markdownu STePSEnHECs-PaCt**

Studenti: Bc. Trong Quoc Huy Dinh, Bc. Břetislav Milota,  
Bc. Martin Reich, Bc. Eliška Toncarová  
Osobní čísla: A25N0040P, A25N0052P, A25N0056P, A25N0061P  
Emaily: qhdt@students.zcu.cz, bmilota@students.zcu.cz, reichm@students.zcu.cz,  
eliska@toncar.cz  
Datum: 16. 5. 2026

# Obsah

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Úvod</b>                                     | <b>2</b>  |
| <b>2</b> | <b>Použité technologie</b>                      | <b>2</b>  |
| <b>3</b> | <b>Architektura aplikace</b>                    | <b>2</b>  |
| 3.1      | Diagramy                                        | 2         |
| 3.2      | Moduly                                          | 3         |
| 3.2.1    | Core                                            | 3         |
| 3.2.2    | Ui                                              | 5         |
| 3.2.3    | Utils                                           | 6         |
| <b>4</b> | <b>Validace markdown souborů</b>                | <b>7</b>  |
| 4.1      | Princip validace                                | 7         |
| 4.2      | Přehled validačních pravidel                    | 8         |
| <b>5</b> | <b>Uživatelské rozhraní</b>                     | <b>9</b>  |
| 5.1      | Hlavní rozvržení okna                           | 9         |
| 5.2      | Pracovní pohled Markdown (Markdown Scene)       | 10        |
| 5.3      | Pracovní pohled Git (Git Scene)                 | 10        |
| <b>6</b> | <b>Testování</b>                                | <b>10</b> |
| 6.1      | Jednotkové testy                                | 10        |
| 6.2      | Testovací scénáře                               | 12        |
| <b>7</b> | <b>Další možná rozšíření</b>                    | <b>12</b> |
| <b>8</b> | <b>Návod k buildu</b>                           | <b>13</b> |
| 8.1      | Požadavky                                       | 13        |
| 8.2      | Koncový uživatel: spuštění programu             | 13        |
| 8.3      | Sestavení spustitelného souboru (build skripty) | 13        |
| 8.4      | Vývojářské prostředí a závislosti               | 14        |
| 8.5      | Poznámky                                        | 14        |
| <b>A</b> | <b>Přílohy</b>                                  | <b>15</b> |
| A.1      | Diagramy                                        | 16        |
| A.2      | Testovací scénáře                               | 21        |

# 1 Úvod

Markdown analyzer je desktopová aplikace určená k analýze a editaci markdown souborů podle STePSEnHECs-PaCt katalogu vzorů. Aplikace zároveň disponuje Git klientem pro jednodušší ukládání provedených změn na markdown souborech. Aplikace je spustitelná na operačních systémech Windows a MacOS.

## 2 Použité technologie

Tato sekce obsahuje přehled hlavních technologií, knihoven a nástrojů, které byly použity při návrhu a vývoji aplikace.

Programovací jazyk:

- **Python 3.10**

Použité knihovny:

- **PyQt6:** Framework pro tvorbu multiplatformního desktopového rozhraní.
- **Python-Markdown (markdown):** Standardní knihovna pro parsování zdrojového textu a jeho konverzi do HTML pro účely živého náhledu (Live Preview).
- **PyMdown Extensions (pymdownx):** Sada rozšíření pro parser Markdownu, která aplikaci dodává podporu pro moderní syntaxi.
- **Git:** Nativní podpora pro správu verzí v lokálních repozitářích.

## 3 Architektura aplikace

Tato sekce popisuje architekturu aplikace, rozdělení do hlavních modulů a toky dat při zpracování markdown souborů či komunikaci s verzovacím systémem Git.

### 3.1 Diagramy

Následující podsekce shrnují závislosti mezi moduly a tok dat při klíčových činnostech aplikace. Samotné diagramy jsou z důvodu přehlednosti přesunuty do **Přílohy A.1**. Diagramy zobrazení a analýzy zahrnují i vrstvy editoru a analyzátoru, aby byl vidět kontext práce se soubory v celém Markdown pohledu.

#### Diagram závislostí modulů

Diagram 1 (viz Příloha A.1) znázorňuje, jak jsou jednotlivé třídy aplikace organizovány do vrstev a v jakém směru jdou importy. Vstupním bodem je `main.py`, které inicializuje centrální třídu `Application`. Ta drží referenci na všechny manažery (`FileManager` atd.) a na hlavní okno `MainWindow`. Manažeři přes UI komponenty (`MarkdownScene`, `TabWidget`, atd.) volají datovou vrstvu – `MarkdownAnalyzer` se svými validátory, parsery (`MarkdownParser`, `TemplateParser` sdílející `BaseParser`), `FileHelper`, `FileMatcher` a balíček `utils.git`. Šipka  $A \rightarrow B$  znamená, že modul A importuje (závisí na) modulu B; barvy odlišují vstupní bod, manažery, UI moduly a datové moduly.

## Tok dat při Git operacích

Diagramy 2 a 3 (viz Příloha A.1) popisují dva paralelní toky. **Flow A** (diagram 2) reprezentuje uživatelem spouštěné Git operace (*status, fetch, pull, push, export staged*): kliknutí v `GitScene` nebo v menu projde přes `GitManager.start_operation`, který vytvoří `GitOperationWorker` (`QThread`) a v něm spustí příslušnou funkci z balíku `utils.git` nad knihovnou `GitPython`. Výsledek (`GitResult`) se přes signály `operation_output` a `operation_finished` vrací zpět do GUI vlákna a vypisuje se v konzoli `GitScene`.

**Flow B** (diagram 3) ukazuje automatické stagování po uložení souboru: `FileManager` vyvolá `MarkdownAutoStager.stage_file_delayed`, který přes jednorázový `QTimer` (1 s) spustí synchronní `stage_markdown_files` a o výsledku informuje signály `file_staged` nebo `staging_failed`.

## Tok dat při zobrazení markdown souboru

Diagram 4 (viz Příloha A.1) ukazuje cestu od otevření souboru (toolbar, dvojklik v průzkumníku, obnovení session nebo proklik v náhledu) až do dvou souběžných pohledů. `FileManager.load_markdown_file` deleguje validaci a načtení bytů na `FileHelper` (s detekcí kódování) a předá obsah do `TabManager.load_file_into_tab`. Z něj tečou data dvěma směry: do `MarkdownEditorWidget` (kde `MarkdownHighlighter` provádí zvýraznění syntaxe po blocích a vykresluje *editor pane*) a – přes signál `textChanged` a `EditorManager.update_live_preview` – do knihovny `markdown` s rozšířeními `pymdownx`, jejíž HTML výstup je obalen výchozím CSS a zobrazen v `QTextBrowser` jako *preview pane*.

## Tok dat při analýze a validaci

Diagram 5 (viz Příloha A.1) ilustruje proces validace, který se spouští po načtení nebo uložení markdown souboru. `FileManager.analyze_current_file` paralelně získá dvě rozdílné věci: `ParsedDocument` z `MarkdownParser.parse_content` a `TemplateRules` pro správnou šablonu, kterou `FileMatcher` dohledá podle rodičovského adresáře souboru. Následně obě data putují do `MarkdownAnalyzer.validate_structure`, který spustí čtyři skupiny kontrol (`FormattingValidator`, `ContentValidator`, vlastní kontroly H1 a sekcí, `LinkValidator`). Výsledný `analysis dict (warnings + passed)` je převeden metodou `generate_report` na text a `TabManager.set_analysis` ho po řádcích zobrazí v panelu `analyzer_list` jako barevně odlišené `QListWidgetItem` pod editorem a náhledem.

## 3.2 Moduly

Tato sekce popisuje logické a adresářové členění zdrojového kódu aplikace. Architektura projektu je navržena modulárně, s jasným oddělením uživatelského rozhraní (prezentační vrstvy) od aplikační logiky a pomocných nástrojů.

### 3.2.1 Core

Modul `core` tvoří samotné jádro celé aplikace. Je zde soustředěna veškerá business logika, správa stavu, propojování komponent a analytické nástroje.

- **analyzer** – balíček obsahuje veškerou logiku pro strukturální, obsahovou a formátovací kontrolu markdown souborů proti pravidlům šablon `STePSEnHECs-PaCt`. Vstupem ana-

lyzátoru je dvojice `ParsedDocument` (analyzovaný soubor) a `TemplateRules` (odpovídající šablona); výstupem je seznam varování (`warnings`) a úspěšně provedených kontrol (`passed`), které `TabManager` promítá do panelu `Analyzer Output`.

– **MarkdownAnalyzer** (`core/analyzer/markdown_analyzer.py`):

Hlavní orchestrátor analýzy. Postupně spouští validátory (`FormattingValidator`, `ContentValidator`, `LinkValidator`) a sám provádí kontroly úrovně dokumentu: počet H1 nadpisů, shodu H1 se jménem souboru, shodu prefixu H1 se šablonou, přítomnost a pořadí povinných sekcí, detekci sekcí, které šablona nezná, a kontrolu správné úrovně nadpisů. Dále validuje breadcrumb řádek dokumentu (počet částí, fixní hodnoty, existenci souborů odkazovaných z breadcrumbs). Pro speciální dokumenty (`template-publication`, in-memory šablony pro *facets* a *publications* indexy) vybrané kontroly přeskakuje. Výsledky umí převést do textového reportu metodou `generate_report()`.

– **ContentValidator** (`core/analyzer/content_validator.py`):

Kontroluje obsah pod jednotlivými nadpisy. Ověřuje, že povinné sekce nejsou prázdné, že množina nalezených typů obsahu (`found_types`) je nadmnožinou typů očekávaných šablonou (`expected_types`), že hlavičky tabulek odpovídají šabloně, že bullet listy obsahují požadované prefixy (např. (+), (-)) a že tabulka existuje tam, kde to šablona vyžaduje. U seskupených sekcí (A–Z bloky) porovnává množinu skutečně přítomných písmen s tou, kterou šablona definuje. Detekuje také kurzívní instrukce přebrané ze šablony (např. *remove or replace this text*), které ve výsledném dokumentu nemají co dělat.

– **FormattingValidator** (`core/analyzer/formatting_validator.py`):

Soustředí kontroly nízkourovňového formátování textu. Hlídá liché počty hvězdiček (neuzavřený bold `**` nebo italic `*`, bullet markery jsou z řádku odfiltrovány), obrázky bez alt textu (`! [] (url)`), správnou skladbu tabulek a přítomnost Unicode *replacement* znaku (`\ufffd`), který indikuje poškozené kódování souboru.

– **LinkValidator** (`core/analyzer/links_validator.py`):

Validuje markdown odkazy na úrovni celého projektu pomocí globálního indexu souborů (`project_index`). Ověřuje, že každý relativní odkaz na `.md` soubor existuje v projektu, že vzájemné odkazy mezi vzory v sekci *Related Patterns* jsou reciproční (pokud vzor A odkazuje na vzor B, musí B odkazovat zpět na A) a že popisek odkazu odpovídá názvu cílového souboru nebo některému z jeho aliasů uvedených v sekci *Also Known As*. Pro indexové šablony lze kontrolu aliasů přeskočit přepínačem `skip_alias_check`.

- **managers** – Tento balíček slouží jako hlavní řídicí (controller) vrstva aplikace. Třídy v této složce oddělují aplikační logiku od samotného uživatelského rozhraní (UI). Jejich úkolem je koordinace procesů, zpracování událostí z UI a komunikace mezi vizuálními komponentami a nízkourovňovými nástroji (jako je analyzátor, Git nebo souborový systém).

– **EditorManager** (`core/managers/editor_manager.py`):

Má na starosti řízení samotného textového editoru a jeho doplňkových vizuálních panelů. Zpracovává změny stavu přepínačů pro zobrazení živého náhledu a výstupu analyzátoru. Jeho klíčovou funkcí je dynamické převádění surového markdownu

do formátovaného HTML kódu pomocí knihovny `markdown` a široké sady rozšíření `pymdownx`. Přebírá obsah z aktuální záložky (`TabManager`) a zajišťuje jeho okamžitou vizuální reprezentaci v panelu `Live Preview`.

– **ErrorManager** (`core/managers/error_manager.py`):

Centralizuje zobrazení vyskakovacích dialogů a chybových hlášení napříč celou aplikací. Pomocí třídních (statických) metod poskytuje jednotné rozhraní pro vyvolání informačních zpráv, varování, kritických chyb či interaktivních dotazů (jako je volba uložení/zahození změn při zavírání aplikace). Všechna tato upozornění interně realizuje prostřednictvím standardní UI komponenty `QMessageBox`.

– **FileManager** (`core/managers/file_manager.py`):

Koordinuje práci s dokumenty v uživatelském rozhraní: načítání šablon z adresáře, otevírání a ukládání markdown souborů ve spolupráci se `TabManager`, spouštění analýzy přes `MarkdownAnalyzer` a obsluhu průzkumníka souborů. Pro čtení, zápis a dialogy deleguje na `FileHelper`. Propojení s oknem, menu a záložkami je navázáno z třídy `Application` (`core/application.py`), včetně zpracování kliknutí na odkazy v náhledu.

– **GitManager** (`core/managers/git_manager.py`):

Slouží jako správce pro operace s verzovacím systémem Git napojený na uživatelské rozhraní. Aby nedocházelo k zamrznutí aplikace při síťových nebo diskových operacích, deleguje spouštění příkazů (např. `push`, `pull`, `commit`) na asynchronní pracovní vlákno `GitOperationWorker`. O výsledcích, průběhu a chybách následně informuje UI pomocí systému událostí (signálů).

– **TabManager** (`core/managers/tab_manager.py`):

Spravuje životní cyklus záložek v (`MarkdownScene`). Pomocí pomocné datové třídy `TabState` udržuje kontext o každé otevřené záložce, včetně cesty k načtenému souboru a indikace neuložených změn (tzv. *dirty state*, reprezentovaný hvězdičkou v názvu). Zajišťuje logiku otevírání, zavírání a přepínání záložek, předává textové změny k dalšímu zpracování a rovněž propisuje formátované výsledky z analyzátoru do odpovídajícího UI panelu v konkrétní záložce.

- **application.py** – Třída `Application` představuje centrální bod celé aplikace. Slouží k inicializaci všech klíčových komponent (hlavní okno, analyzátor, manažeři, pomocné nástroje) a k propojení uživatelského rozhraní s aplikační logikou. Zajišťuje propojení událostí z UI (jako jsou kliknutí na tlačítka, výběr z menu či změny zaškrťovacích políček) s konkrétními akcemi v jednotlivých manažerech. Dále řídí životní cyklus aplikace, což zahrnuje aplikaci grafického motivu (QSS), spuštění hlavní smyčky událostí, bezpečné načítání předchozího stavu při startu (např. pozice okna, otevřené záložky, poslední adresář) a jeho ukládání při ukončování aplikace.

### 3.2.2 Ui

Modul `ui` obsahuje definici uživatelského rozhraní aplikace:

- **assets** – aktiva aplikace (ikony a QSS vzhledy.)
- **components** – ui komponenty použité v jednotlivých scénách.

- **scenes** – scény aplikace.
- **main\_window.py** – definice hlavního okna aplikace.

### 3.2.3 Utils

Modul `utils` obsahuje pomocné nástroje aplikace.

- **FileHelper** (`utils/file_helper.py`):

Třída `FileHelper` soustředí nízkoúrovňové operace se soubory a systémovými dialogy PyQt6. Poskytuje výběr jednoho či více markdown souborů a pracovního adresáře (`QFileDialog`), validaci existence souboru, typu a oprávnění ke čtení, čtení a zápis s detekcí kódování (UTF-8, případně knihovna `charset_normalizer`, záložní CP1252) a zapamatování kódování pro konzistentní uložení. Dále vyhledává markdown soubory v adresáři (s možností rekurze), hlídá rozumnou velikost obsahu podle `FileConstants` a pomáhá při rozlišení relativních odkazů z náhledu (`resolve_relative_markdown_link`).

- **Git operace** (`GitManager`, balík `utils/git`):

Správu příkazů Git zajišťuje `GitManager` (`core/managers/git_manager.py`), který ve vlastním vlákně spouští `GitOperationWorker` odvozený od `QThread`. V jeden okamžik může běžet nejvýše jedna operace; další požadavky jsou odmítnuty s hláškou pro uživatele. Pracovník podle typu operace volá funkce z balíku `utils/git` (stav repozitáře, `fetch`, `pull`, `push` včetně `commit` zprávy, `staging`, `export staged` změn do archivu apod.), přičemž práce s repozitářem probíhá přes knihovnu `GitPython`. Výsledky jsou předávány jako `GitResult`. Nebezpečná akce exportu vyžaduje potvrzení dialogem o možné ztrátě dat. Signály manažeru aktualizují `Git Scene` (konzole, povolení ovládacích prvků); konkrétní propojení je v `Application`.

- **Parsery** (balík `utils/parsers`):

Balík `utils/parsers` převádí surový text markdown souborů a šablon na strukturované datové třídy, se kterými následně pracuje analyzátor. Aby se logika rozpoznávání nadpisů, `breadcrumbs` a typů obsahu neopakovala ve dvou modulech, je veškerá regexová a klasifikační logika soustředěna do sdíleného modulu `base_parser`.

- **base\_parser** (`utils/parsers/base_parser.py`):

Modul sdílených parsovacích primitiv. Obsahuje funkce `parse_heading_line()` (rozparsuje řádek nadpisu na úroveň, čistý text, odkaz a u šablon i indikátory *optional* a italicických *variable* placeholderů), `split_h1()` (rozdělí H1 na prefix a hodnotu podle dvojtečky), `parse_breadcrumbs()` (rozpozná breadcrumb řádek od italic instrukcí šablony) a `classify_content_line()` (klasifikuje řádek obsahu na typy text, bullet list, tabulka, obrázek, odkaz, horizontální čára nebo footnote a extrahuje hlavičky tabulky a bullet prefixy). Všechny ostatní parsery pracují právě nad těmito funkcemi.

- **MarkdownParser** (`utils/parsers/markdown_parser.py`):

Převádí načtený markdown soubor na `ParsedDocument`. Výstupem je dvojice `DocumentMeta` (cesta k souboru, `breadcrumbs` v originálním i normalizovaném tvaru, prefix a hodnota H1) a seznam `HeadingInfo` (úroveň nadpisu, text, číslo řádku a vnořené `ContentInfo` s nalezenými typy obsahu, bullet prefixy, hlavičkami

tabulky a tzv. *raw\_lines* pro pozdější kontroly formátování). Souvislé běhy A–Z nadpisů jsou automaticky sbaleny do jediného `is_group=True` záznamu, aby je bylo možné porovnávat se šablonou jako celek. Metoda `get_link_metadata()` navíc z dokumentu vytáhne aliasy (sekce *Also Known As*) a vzájemné odkazy (*Related Patterns*); výsledky se promítají do globálního `project_index`, nad kterým pracuje `LinkValidator`.

– **TemplateParser** (`utils/parsers/template_parser.py`):

Třída `TemplateParser` představuje Analogie `MarkdownParser` pro šablonové soubory. Výstupem je `TemplateRules` obsahující `DocumentRules` (očekávané breadcrumbs, prefix H1) a seznam `HeadingRules` s vnořenými `ContentRules` (očekávané typy obsahu, povinné bullet prefixy, hlavičky tabulek). Parser navíc detekuje markery (*optional*) a italské *variable* placeholderů v nadpisech a stejně jako dokumentový parser sbaluje A–Z bloky. Načtené šablony jsou udržovány v interní mapě `templates` a vyhledávány podle jména.

– **FileMatcher** (`utils/file_matcher.py`):

Třída `FileMatcher` páruje markdown soubor s odpovídající šablonou na základě jména rodičovského adresáře (např. soubory v adresáři `patterns/` se validují proti šabloně `template-pattern`). Speciální případy jako indexy *facets* a publikační soubory jsou ošetřeny pomocí in-memory šablon z modulu `facets_index_template`.

– **facets\_index\_template** (`utils/facets_index_template.py`):

Definuje `TemplateRules` programaticky, bez načítání ze souboru, pro dokumenty, které nemají odpovídající `.md` šablonu na disku (indexy podle facet, seznam publikací). Tyto šablony mají volnější pravidla – povolují variabilní H1 a libovolný textový obsah.

– **Konstanty a výjimky** (`utils/constants.py`, `utils/exceptions.py`):

Soubor `constants.py` centralizuje všechny tzv. magické hodnoty – regulární výrazy pro nadpisy, odkazy a breadcrumbs, povolené přípony souborů, limity velikosti, ikony pro report (`ICON_OK`, `ICON_WARNING`) i konstanty pro Git operace. Soubor `exceptions.py` definuje hierarchii vlastních výjimek (`BaseAppException`, `GitException`, `FileNotFoundError`, `InvalidInputError`, ...) s podporou srozumitelných hlášek a titulků dialogů, díky čemuž `ErrorManager` dokáže zobrazit konzistentní chybové zprávy.

## 4 Validace markdown souborů

Validace markdown souborů je hlavní funkcí aplikace. Cílem je ověřit, že každý dokument katalogu STePSEnHECs-PaCt odpovídá pravidlům příslušné šablony – má správně sestavený nadpis, breadcrumbs, sekce v očekávaném pořadí, smysluplný obsah pod každou sekcí a funkční vzájemné odkazy mezi vzory. Tok dat při validaci je znázorněn na obrázku 5.

### 4.1 Princip validace

Postup validace je rozdělen do tří kroků:



1. **Parsování dokumentu a šablony.** `MarkdownParser` převede obsah markdown souboru na `ParsedDocument` (metadata, seznam nadpisů a strukturovaný obsah pod každým z nich). `TemplateParser` převede odpovídající šablonu na `TemplateRules`. Spárování dokumentu se šablonou zajišťuje `FileMatcher` na základě jména rodičovského adresáře (např. `patterns/` → `template-pattern`); pro indexové soubory se použijí in-memory šablony z modulu `facets_index_template`.
2. **Spuštění dílčích validátorů.** `MarkdownAnalyzer` jako orchestrátor postupně spustí třídu `FormattingValidator` (formátování textu), `ContentValidator` (obsah jednotlivých sekcí), kontroly struktury a breadcrumbs ve vlastním těle a nakonec `LinkValidator` (vzájemné odkazy v rámci celého projektu, pokud je k dispozici globální `project_index`). Každý validátor pracuje nad společnými datovými třídami a vrací dva seznamy: `warnings` (nalezené chyby) a `passed` (úspěšně splněné kontroly).
3. **Agregace a zobrazení výsledků.** Výsledky všech validátorů jsou sloučeny, seřazeny podle čísla řádku a předány `TabManager`, který je zobrazí v panelu `Analyzer Output` s barevným odlišením úspěchů, varování a chyb.

## 4.2 Přehled validačních pravidel

Následující tabulka shrnuje všechna kontrolovaná pravidla, sloupec *Kde se implementuje* odkazuje na umístění kontroly v kódu: `document_loader` = během parsování v `MarkdownParser`, *validátor* = některý z validátorů v balíku `core/analyzer`, *oboje* = pravidlo je z části detekováno už při parsování (uložení potřebných dat do `ParsedDocument`) a finálně vyhodnoceno validátorem.

| #  | Kategorie           | Co se ověřuje                                                        | Kde se implementuje          |
|----|---------------------|----------------------------------------------------------------------|------------------------------|
| 1  | Struktura dokumentu | Dokument musí mít právě jeden H1 nadpis.                             | <code>document_loader</code> |
| 2  | Struktura dokumentu | <code>dokument.h1.value</code> (case-insensitive) == stem souboru.   | <code>document_loader</code> |
| 3  | Struktura dokumentu | Prefix v H1 musí souhlasit s tím, co šablona definuje.               | oboje                        |
| 4  | Struktura dokumentu | Všechny non-optional headingy ze šablony musí existovat v dokumentu. | oboje                        |
| 5  | Struktura dokumentu | Pokud optional sekce existuje, její text musí odpovídat šabloně.     | oboje                        |
| 6  | Struktura dokumentu | Sekce musí být ve stejném pořadí jako v šabloně.                     | oboje                        |
| 7  | Struktura dokumentu | Dokument neobsahuje headingy, které šablona vůbec nezná.             | oboje                        |
| 8  | Struktura dokumentu | Sekce musí mít správnou úroveň (#, ##, ###... ) podle šablony.       | oboje                        |
| 9  | Breadcrumbs         | Dokument musí mít breadcrumb řádek.                                  | <code>document_loader</code> |
| 10 | Breadcrumbs         | Počet částí oddělených ' <code>¿</code> ' musí odpovídat šabloně.    | oboje                        |
| 11 | Breadcrumbs         | Fixní části (ne poslední) musí textově odpovídat šabloně.            | oboje                        |

| #  | Kategorie   | Co se ověřuje                                                                                                                                        | Kde se implementuje |
|----|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| 12 | Breadcrumbs | Každý [label] (url) v breadcrumbs odkazuje na existující soubor.                                                                                     | validátor           |
| 13 | Obsah sekcí | Non-optional sekce musí mít alespoň nějaký obsah.                                                                                                    | oboje               |
| 14 | Obsah sekcí | found_types musí být nadmnožinou expected_types (kromě 'text').                                                                                      | oboje               |
| 15 | Obsah sekcí | Sloupce tabulky musí odpovídat šabloně.                                                                                                              | oboje               |
| 16 | Obsah sekcí | Pokud šablona definuje bullet_prefixes, dokument je musí obsahovat.                                                                                  | oboje               |
| 17 | Obsah sekcí | V dokumentu nesmí zůstat kurzívní instrukce ze šablony (např. 'remove or replace. . .').                                                             | oboje               |
| 18 | Obsah sekcí | Dokument musí mít kompletní sadu písmen dle šablony.                                                                                                 | oboje               |
| 19 | Obsah sekcí | Tabulka existuje, ale může být prázdná.                                                                                                              | oboje               |
| 20 | Formátování | Liché počty ** na řádku = neuzavřený bold.                                                                                                           | oboje               |
| 21 | Formátování | Liché počty * (mimo bullet marker) = neuzavřená italic.                                                                                              | oboje               |
| 22 | Formátování | ! [] (url) bez alt textu = přístupnostní chyba.                                                                                                      | oboje               |
| 23 | Formátování | Přítomnost replacement character (□, \ufffd) = špatné kódování souboru.                                                                              | oboje               |
| 24 | Formátování | Správně zadaná tabulka.                                                                                                                              | oboje               |
| 25 | Linky       | [] (url) nebo [label] () = špatně zadaný odkaz.                                                                                                      | document_loader     |
| 26 | Linky       | [label] (relativní_cesta) musí odkazovat na existující soubor.                                                                                       | document_loader     |
| 27 | Linky       | Pokud pattern A odkazuje v sekci 'Related Patterns' na pattern B, pak B musí mít odkaz zpět na A.                                                    | validátor           |
| 28 | Linky       | Pokud je v A–Z sekci odkaz na pattern pod jiným názvem (alias), ověřit, že cílový soubor existuje a cílový pattern má alias v sekci 'Also Known As'. | validátor           |

Tabulka 1: Přehled validačních pravidel aplikovaných na markdown soubory.

## 5 Uživatelské rozhraní

Aplikace plně využívá vlastní tmavý motiv (Dark Mode) a vektorové ikony. Rozhraní je rozděleno do dvou hlavních pracovních pohledů (scén), mezi kterými lze přepínat.

### 5.1 Hlavní rozvržení okna

- **Postranní panel (Sidebar):** Úzký svislý panel na levé straně okna, který slouží k rychlému přepínání mezi editorem Markdownu a rozhraním pro správu Gitu.
- **Horní menu a lišta (Toolbar):** Poskytuje přístup k nejpoužívanějším akcím (otevírání souborů a složek, ukládání, spouštění Git operací) a obsahuje klávesové zkratky.

## 5.2 Pracovní pohled Markdown (Markdown Scene)

Tento pohled je primárním rozhraním pro úpravu a analýzu markdown souborů.

- **Průzkumník souborů (File Explorer):** Levý postranní panel zobrazující stromovou strukturu načteného pracovního adresáře. Umožňuje rychlou navigaci a otevírání dokumentů dvojitým kliknutím. Pro maximalizaci pracovního prostoru lze panel skrýt.
- **Správce záložek (Tab Manager):** Centrální pracovní plocha umožňující editaci více souborů současně. Záložky vizuálně indikují neuložené změny (hvězdičkou) a chrání uživatele před ztrátou dat při zavření okna.
- **Ovládací panel editoru:** Obsahuje rychlé přepínače (checkboxy) pro zobrazení nebo skrytí živého náhledu a výstupu analyzátoru.
- **Živý náhled (Live Preview):** Panel umístěný napravo od textového editoru, který v reálném čase vykresluje formátovaný HTML výstup (včetně syntaxe kódu, tabulek a odkazů) na základě psaného textu.
- **Výstup analyzátoru (Analyzer Output):** Spodní panel v rámci záložky, který interaktivně vypisuje výsledky strukturální kontroly a validace odkazů. Barvami (červená, žlutá nebo zelená) odlišuje úspěšné kontroly, varování a kritické chyby.

## 5.3 Pracovní pohled Git (Git Scene)

Tento pohled slouží k integraci s verzovacím systémem a poskytuje grafické rozhraní pro správu repozitáře.

- **Vstupní pole pro zprávu:** Víceřádkové textové pole, které vynucuje zadání zprávy ke commitu před odesláním změn.
- **Ovládací panel Gitu:** Tlačítka pro spouštění asynchronních git operací (Status, Fetch, Pull, Push, Export Staged).
- **Konzole (Output Console):** Textový výpis umístěný uprostřed, který uživateli poskytuje přímou zpětnou vazbu o probíhajících Git operacích a loguje automatické přidávání souborů (auto-staging) běžící na pozadí.

# 6 Testování

Tato sekce popisuje proces a metodiku ověřování správné funkčnosti aplikace.

## 6.1 Jednotkové testy

Jednotkové testy desktopové aplikace jsou napsané v frameworku `pytest` a nacházejí se v adresáři `application/tests/`. Testy jsou tematicky rozděleny do podadresářů `analyzers/`, `parsers/`, `file/` a `git/`, které kopírují strukturu produkčního kódu. Sdílené fixtures (např. inicializace prázdného Git repozitáře s počátečním commitem) jsou definovány v `conftest.py` a `tests/fixtures/`.

Testy se spouští z adresáře `application/` příkazem `pytest tests/` (případně s přepínači `-v` pro podrobný výpis nebo `--cov=core --cov=utils` pro měření pokrytí). Každý test je

nezávislý – využívá pytest fixtures `tmp_path` pro dočasné soubory, takže testy nezasahují do skutečného souborového systému ani Git repozitáře projektu.

### Testy analyzátoru (`tests/analyzers/`)

- **test\_markdown\_analyzer.py** – pokrývá orchestrátor `MarkdownAnalyzer`: inicializaci s vlastním i výchozím `base_path`, kontroly počtu H1 nadpisů, shody H1 se jménem souboru, prefixu H1, povinných sekcí, pořadí sekcí, detekce neznámých sekcí, úrovní nadpisů, validace breadcrumbs a jejich odkazů, vyhledávání markdown odkazů a generování textového reportu.
- **test\_content\_validator.py** – pokrývá kontroly obsahu sekcí: prázdné povinné sekce, shodu `found_types` s `expected_types`, sloupce a existenci tabulek, povinné bullet prefixy, detekci kurzívních instrukcí přebraných ze šablony a kontroly úplnosti A–Z bloků.
- **test\_formatting\_validator.py** – pokrývá nízkoúrovňové kontroly formátování: neuzavřený bold a italic, obrázky bez alt textu, korektnost tabulek a detekci Unicode *replacement* znaku.
- **test\_links\_validator.py** a **test\_relative\_checking.py** – pokrývají třídu z core modulu `LinkValidator`: kontrolu existence cílů relativních odkazů, reciproční odkazy mezi vzory v sekci *Related Patterns* a konzistenci aliasů se sekci *Also Known As*.

### Testy parserů (`tests/parsers/`)

- **test\_base\_parser.py** – pokrývá sdílené parsovací primitivy: `parse_heading_line()` (úrovně nadpisů, optional a variable markery, odkazy v nadpisech), `split_h1()`, `parse_breadcrumbs()` (rozlišení od italic instrukcí) a `classify_content_line()` pro všechny typy obsahu (text, bullet, tabulka, obrázek, odkaz, footnote, horizontal rule).
- **test\_markdown\_parser.py** – pokrývá `MarkdownParser`: sestavení `ParsedDocument` z různých vstupů, korektní vyplnění `DocumentMeta` (breadcrumbs, prefix a hodnota H1), sbalení A–Z bloků do skupinových nadpisů a extrakci aliasů a vzájemných odkazů metodu `get_link_metadata()`.
- **test\_template\_parser.py** – pokrývá `TemplateParser`: rozpoznání povinných a optional sekcí, italicických *variable* placeholderů, pravidel obsahu (`ContentRules`), správné sestavení `HeadingRules` a sbalení A–Z bloků v šabloně.

### Testy souborové vrstvy (`tests/file/`)

- **test\_paths.py** – ověřuje výpočet kořenové cesty repozitáře v `Config` (vyhledání adresáře s `.git` v aktuálním nebo nadřazených adresářích, chyba pokud Git repozitář neexistuje), výchozí cestu k markdown souboru a inicializaci `MarkdownAnalyzer` vůči parametru `base_path`.
- **test\_file\_helper.py** – pokrývá `FileHelper`: čtení a zápis s UTF-8 a CP1252 (roundtrip), zamítnutí uložení nekompatibilních znaků při souboru s ním nastaveným CP1252, `find_markdown_files` (s rekurzí i bez deduplikace), `get_file_info`, `resolve_relative_markdown`, validaci souborů, limity velikosti (`MAX_FILE_SIZE_MB`) a čtení/zápis včetně výjimek (`FileNotFoundError`, `FileSizeError`, `FileWriteError`).

- **test\_file\_matcher.py** – pokrývá `FileMatcher`: přiřazení šablony podle rodiče (či prarodiče) adresáře v cestě k souboru, složky katalogu (`catalogue/`, `publications`, `categories` metodiky apod.), indexové soubory pod `facets/` (šablony z `facets_index_template`), pády pro neznámé cesty a cesty se zpětným lomítkem (`\`).

### Testy Git operací (`tests/git/`)

- **test\_git\_runner.py** – testuje pomocné funkce v `utils/git/runner.py`: normalizaci cesty vůči kořeni repozitáře, rozpoznání markdown přípon, nalezení kořene `.git` ze zanoření, filtrovaný Git status jen pro Markdown, formát výpisu statusu pro čistý i měněný working tree včetně staged změn.
- **test\_git\_operations.py** – pokrývá funkce pod `utils/git/` a chování `GitManager`: staging Markdown souborů a filtrování případně zadaných cest (`stage_markdown_files`, `stage_file`), odstaging (`unstage_all`), `commit` (prázdná či příliš dlouhá zpráva, žádné staged změny, režim `stage_all`), `push_markdown_changes` (vlastní/z výchozí/z prázdné zprávě použití defaultu, odmítnutí příliš dlouhé commit zprávy, blokáce když lokální větev zaostává za vzdálenou větví při mockovaném synchronizačním stavu), `fetch/pull/push` bez remotu či při problémovém sync stavu, export staged do ZIPu (`export_staged_files_zip`), větev zamítnutí druhé paralelní operace u `GitManager` při aktivním workeru, rozřazení neznámého příkazu a známých operací ve `GitOperationWorker` a větev závislá na uživatelském potvrzení u export staged.

## 6.2 Testovací scénáře

Pro komplexní ověření funkčnosti aplikace z pohledu uživatele, správného chování uživatelského rozhraní, ošetření chyb a integrace verzovacího systému Git byla vytvořena sada manuálních testovacích scénářů. Tyto scénáře pokrývají jak standardní průchody aplikací (tzv. happy paths), tak i nestandardní chování a okrajové případy.

Kompletní výpis všech testovacích scénářů naleznete v **Příloze A.2**.

## 7 Další možná rozšíření

Tato sekce obsahuje seznam dalších možných rozšíření aplikace.

- **Zobrazení struktury souboru** – zobrazení struktury souboru ve formě hierarchie sekcí. Uživatel by měl mít možnost si vybrat, zda se tato hierarchie zobrazí. Proklikem na sekci by se měl editor přesunout na danou sekci v markdown souboru.
- **Vícejazyčná podpora** – podpora více jazyků. V základu podporuje pouze angličtinu.
- **Export do .dot formátu** – export souboru v `.dot` formátu, který by podle odkazů v markdown souborech zobrazil grafovou strukturu těchto spojů.
- **Vytváření nových souborů** – Aplikace v základu pouze upravuje obsah existujících markdown souborů z pevného disku. V budoucnu by mohla mít možnost vytvářet nové markdown soubory.
- **Přesouvání/přejmenovávání souborů a adresářů** – aplikace v základu pouze otevírá existující soubory z pevného disku. V budoucnu by mohla mít možnost přesouvat a

přejmenovávat soubory a adresáře.

- **Vylepšený git commit** – rozdělení řádků markdown souborů do více commitů.

## 8 Návod k buildu

Tato sekce shrnuje instalaci, sestavení a spuštění aplikace. Podrobnější anglické návody jsou přímo v repozitáři a doplňují ji zejména tyto soubory:

- **application/README.md** – uživatelský manuál (ovládání rozhraní, základní tipy k analýze a Git režimu).
- **application/INSTALLATION\_OVERVIEW.md** – rozcestník pro vývojářskou a uživatelskou instalaci.

Koncept odpovídá rozlišení *běžný uživatel* versus *vývojář*: koncový uživatel typicky spouští již sestavený program nebo získá spustitelný soubor z dodaného skriptu; vývojář pracuje se zdrojovým kódem, virtuálním prostředím Pythonu a závislostmi v `application/requirements.txt`. Aplikaci je vhodné spouštět z kořenové složky projektu (obsahuje katalog projektu a metadata Gitu); při špatném pracovním adresáři se program nemusí spustit správně.

### 8.1 Požadavky

- **Git** – pokud se používají funkce integrace s Git, musí být Git nainstalovaný v systému ([git-scm.com/downloads](https://git-scm.com/downloads)). Ověření: příkaz `git --version`.
- **Python 3.10** – nutný pro vývoj a ruční instalaci závislostí; viz níže. Instalační dokumentace: [python.org/downloads](https://python.org/downloads).

### 8.2 Koncový uživatel: spuštění programu

Pokud je k dispozici již sestavený spustitelný soubor, není třeba ručně zakládat virtuální prostředí ani instalovat závislosti z PyPI.

**Windows** – v adresáři s programem nebo ve složce `dist` (PowerShell akceptuje i zápis s dopřednými lomítky):

```
./markdown_analyzer.exe  
./dist/markdown_analyzer.exe
```

**Linux / macOS** – v adresáři s programem nebo ve složce `dist`:

```
./markdown_analyzer  
./dist/markdown_analyzer  
chmod +x ./dist/markdown_analyzer # pokud system spusteni odmitne
```

### 8.3 Sestavení spustitelného souboru (build skripty)

Build skripty leží v **kořeni klonovaného repozitáře** (vedle složky `application/`). Po dokončení sestavení bývá výstup ve složce `dist/`.

**Windows (PowerShell)** – zápis cesty s dopřednými lomítky je ekvivalentní klasickému tvaru s obrácenými lomítky:

```
cd C:/cesta/k/koreni_projektu
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
./build_script.ps1
./dist/markdown_analyzer.exe
```

#### Linux / macOS:

```
cd /cesta/k/koreni_projektu
chmod +x build_script.sh # jednou, pokud skript nema pravo ke spusteni
./build_script.sh
./dist/markdown_analyzer
```

## 8.4 Vývojářské prostředí a závislosti

Pro práci se zdrojovým kódem je doporučené virtuální prostředí Pythonu a instalace balíčků z `application/requirements.txt`.

1. Ověřit verzi interpretu (Python 3.10.x), např. `python3 --version` nebo na Windows `py --version`.
2. Naklonovat repozitář a přejít do složky aplikace:

```
git clone <HTTPS/SSH>
cd .../application
```

3. Vytvořit a aktivovat virtuální prostředí:

```
python3.10 -m venv venv # Linux / macOS
py -3.10 -m venv venv # Windows, pokud chybi python3.10
source venv/bin/activate # Linux / macOS
# Windows (PowerShell): ./venv/Scripts/Activate.ps1
```

4. Nainstalovat závislosti:

```
pip install -r requirements.txt
# alternativa: python -m pip install -r requirements.txt
```

5. Sestavit balík podle předchozí podsekce z **kořene** repozitáře (`build_script.ps1` nebo `build_script.sh`), nebo při ladění spouštět modul z aktivovaného virtuálního prostředí v `application`, např. `python main.py`.

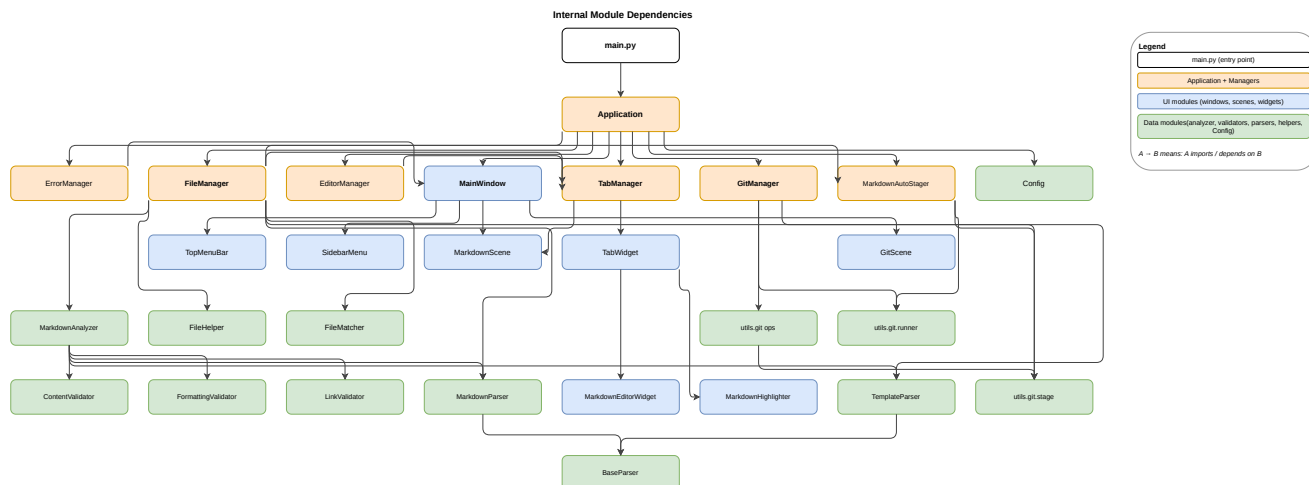
## 8.5 Poznámky

- Starší verze Pythonu než 3.10 mohou způsobit problémy se závislostmi.
- Na Windows může výchozí politika PowerShellu blokovat spouštění skriptů; řešením je dočasné povolení pro proces (viz výše) nebo nastavení podle dokumentace systému.
- Pokud Git funkce selžou, zkontrolujte instalaci Gitu v systému.

## **A Přílohy**



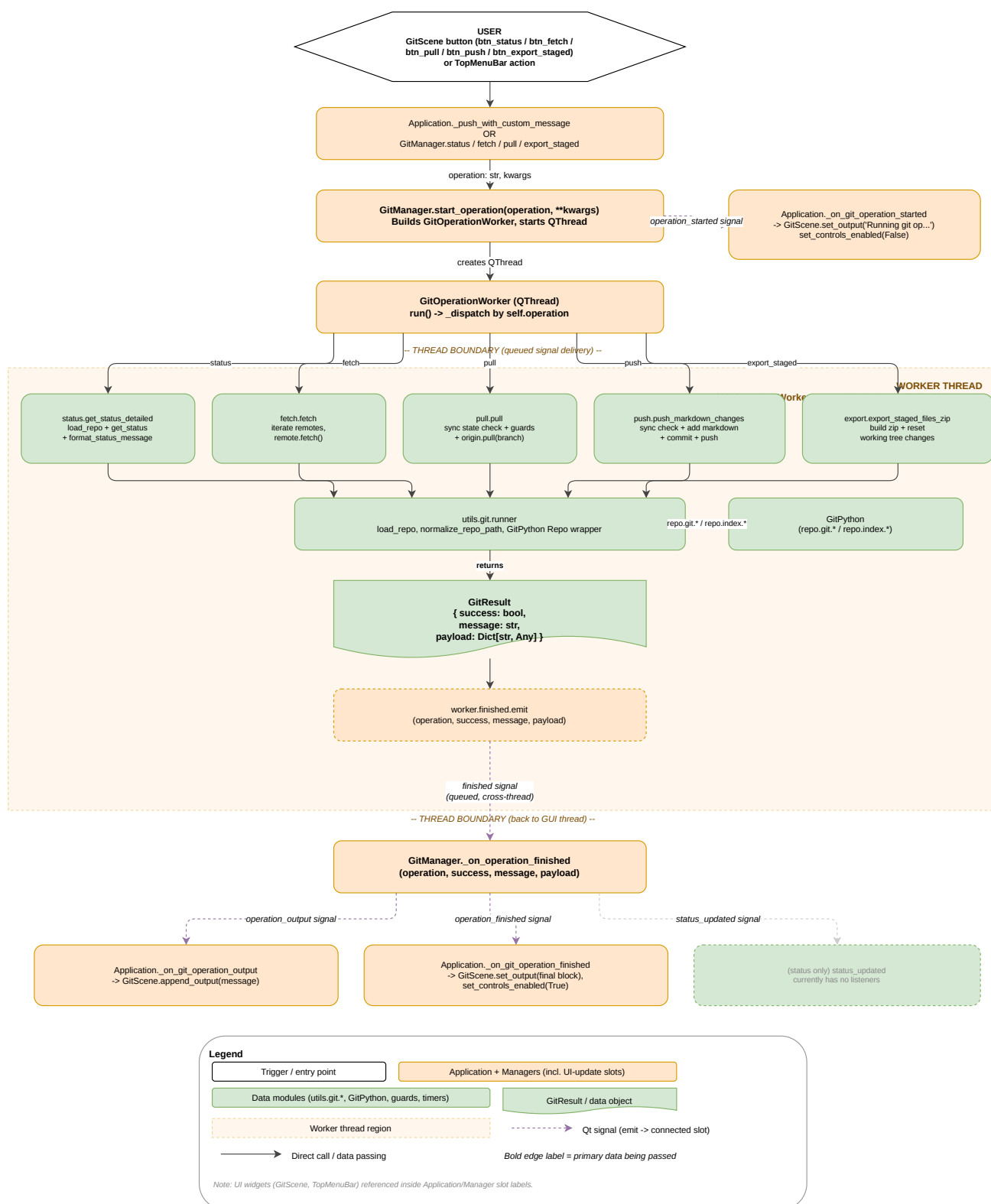
## A.1 Diagramy



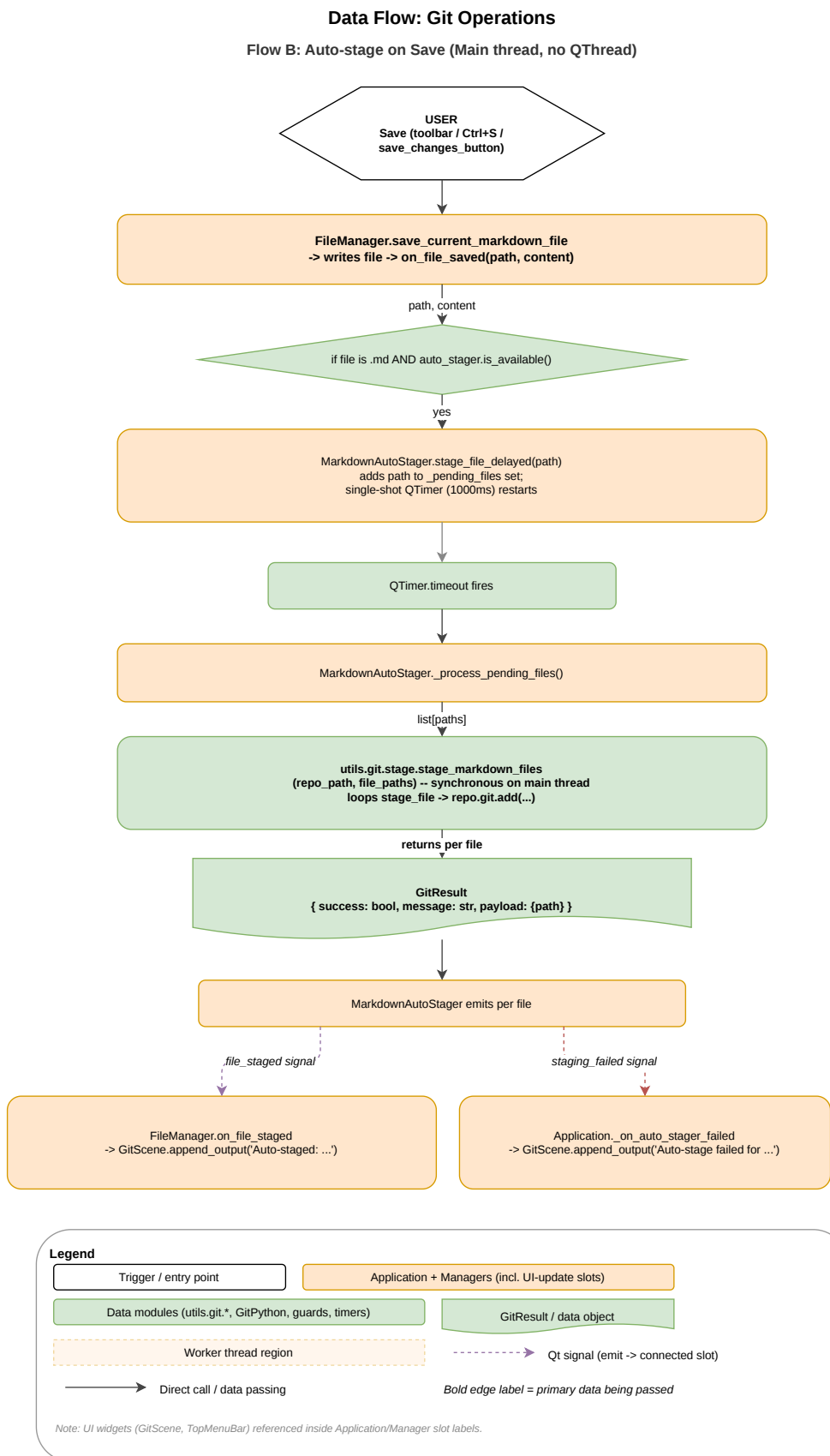
Obrázek 1: Závislosti hlavních modulů aplikace (imports a vrstvy mezi vstupním bodem, aplikací, manažery, uživatelským rozhraním a pomocnými knihovnami).

## Data Flow: Git Operations

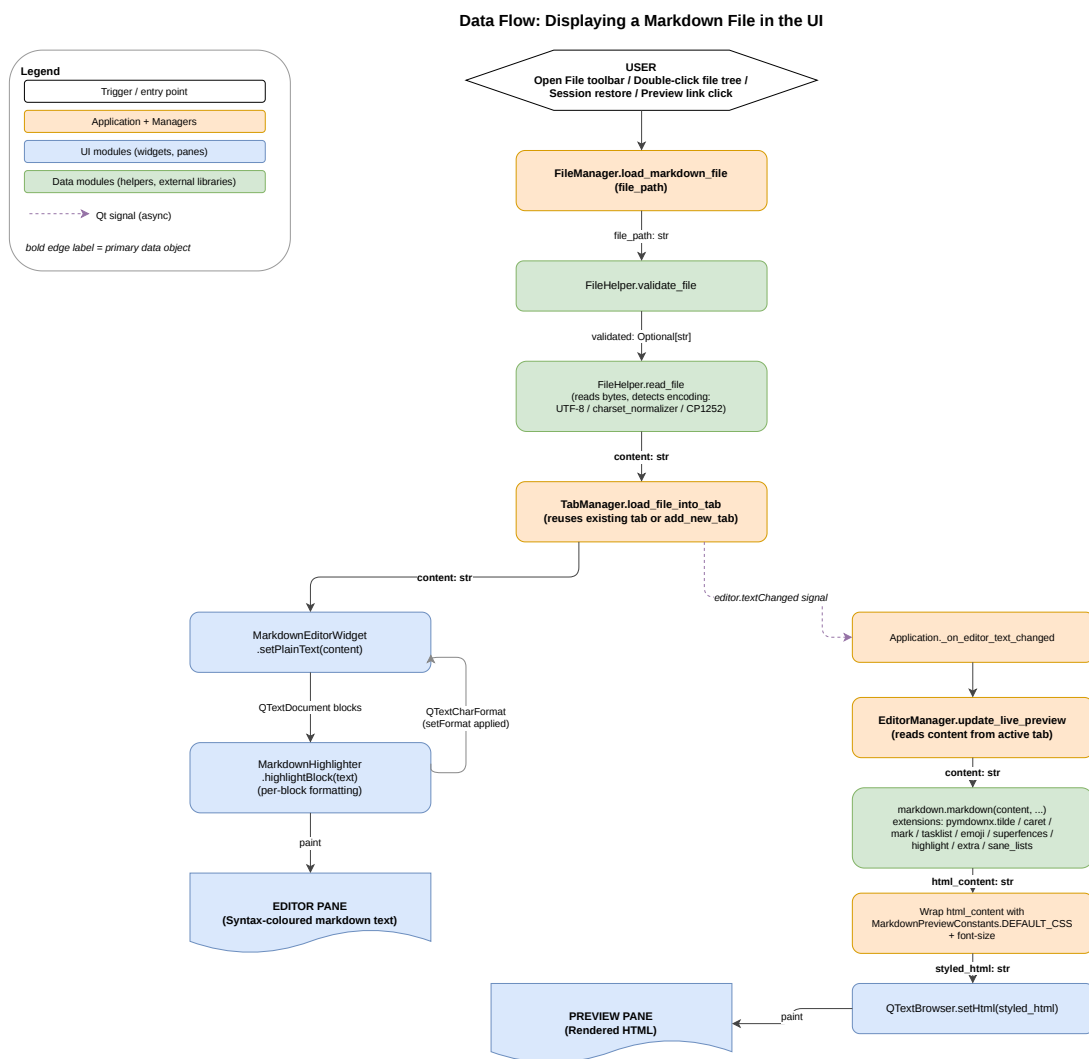
### Flow A: Threaded Git Operations (Status / Fetch / Pull / Push / Export)



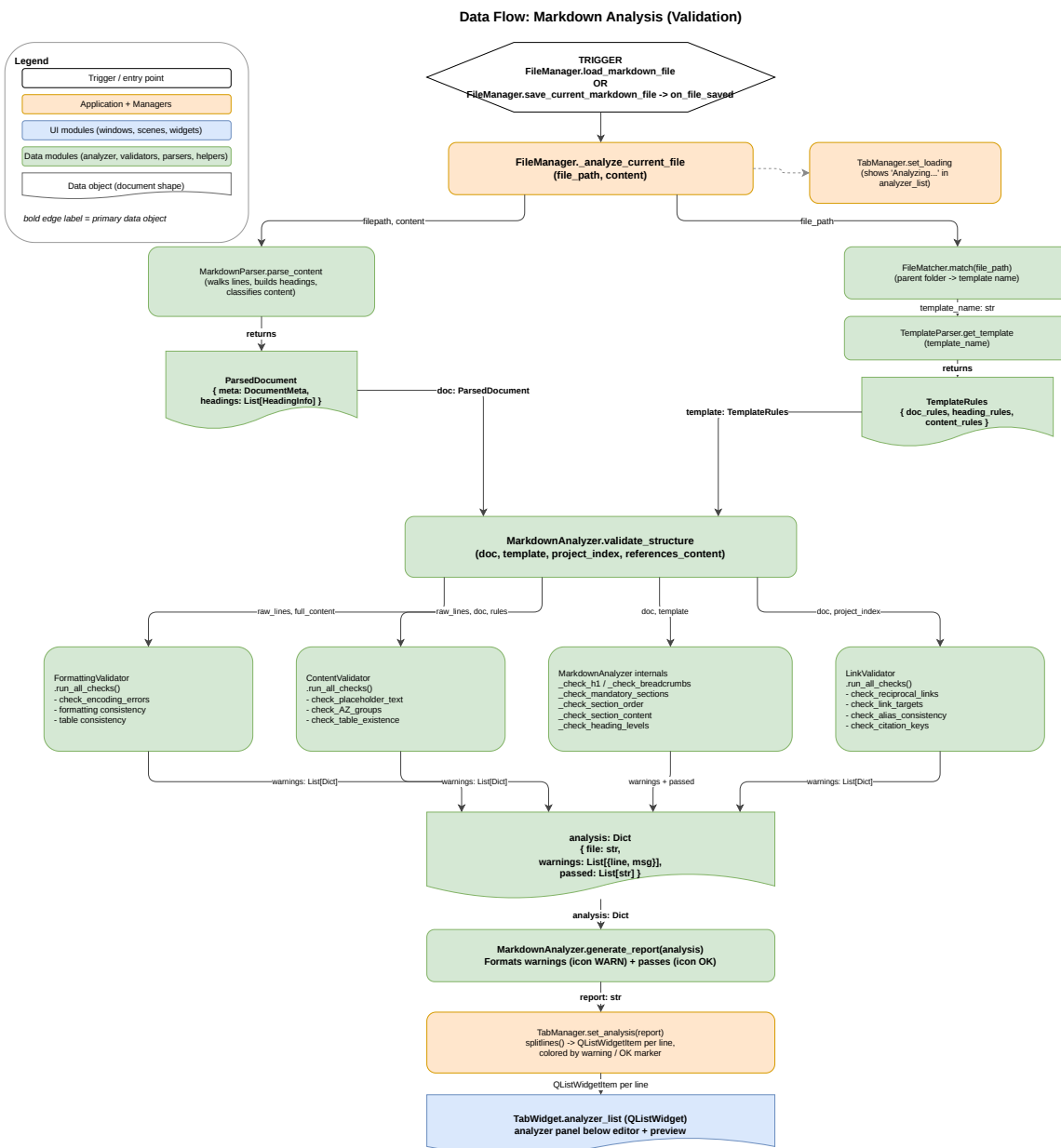
Obrázek 2: Tok dat při Git operacích (Flow A): akce uživatele v Git Scene nebo v menu, orchestrace v GitManager, provedení příkazů ve vlákně GitOperationWorker pomocí balíku utils/git a GitPython, zpětná vazba do konzole rozhraní.



Obrázek 3: Tok dat při Git operacích (Flow B): automatické přidávání změn na pozadí (Auto-stager) spouštěné po uložení souboru.



Obrázek 4: Tok dat při zobrazení markdown souboru: `FileManager` využívá `FileHelper` k validaci a načtení obsahu, `TabManager` obsluhuje záložky a editor; živý náhled vzniká převodem markdownu na HTML a zobrazením v náhledovém widgetu.



Obrázek 5: Tok dat při analýze a validaci struktury markdownu po načtení nebo uložení: od FileManager přes parsování dokumentu, výběr šablony až po validátory a výpis výsledků v panelu analyzátoru.

## A.2 Testovací scénáře

Tento dokument obsahuje hlavní funkční testy pro aplikaci, se zaměřením na uživatelské rozhraní, ošetření chyb a integraci Gitu.

### 1. Operace s Gitem (Git Operations)

#### Scénář 1.1: Provedení Git Push se zadanou commit zprávou

**Popis:** Ověření, že aplikace provede push, pokud uživatel zadá zprávu ke commitu.

- **DÁNO** uživatel se nachází v pohledu Git (Git scene) a má připravené (staged) změny.
- **A** textové pole pro commit zprávu obsahuje text.
- **KDYŽ** uživatel klikne na tlačítko "Push".
- **PAK** operace Git push na pozadí se PROVEDE.

#### Scénář 1.2: Zamezení Git Push s prázdnou commit zprávou

**Popis:** Ověření, že aplikace zablokuje push, pokud uživatel zapomene zadat zprávu ke commitu.

- **DÁNO** uživatel se nachází v pohledu Git (Git scene) a má připravené (staged) změny.
- **A** textové pole pro commit zprávu je zcela prázdné (nebo obsahuje jen mezery).
- **KDYŽ** uživatel klikne na tlačítko "Push".
- **PAK** se zobrazí kritický dialog s titulkem "Missing Commit Message".
- **A** operace Git push na pozadí se NEPROVEDE.

#### Scénář 1.3: Export změněných souborů

**Popis:** Ověření, že uživatel úspěšně exportuje změněné soubory z pracovního adresáře.

- **DÁNO** uživatel se nachází v pohledu Git.
- **KDYŽ** uživatel klikne na tlačítko "Export Staged".
- **PAK** se zobrazí varovný dialog s titulkem "Warning: Potential Data Loss".
- **KDYŽ** uživatel klikne na "Ano"(Yes).
- **PAK** se dialog zavře a operace exportu se provede.

#### Scénář 1.4: Zamezení nechtěné ztráty dat při Git Exportu

**Popis:** Ověření, že je uživatel varován před exportem, protože tato akce vyčistí pracovní adresář.

- **DÁNO** uživatel se nachází v pohledu Git.
- **KDYŽ** uživatel klikne na tlačítko "Export Staged".
- **PAK** se zobrazí varovný dialog s titulkem "Warning: Potential Data Loss".
- **KDYŽ** uživatel klikne na "Ne"(No).
- **PAK** se dialog zavře a operace exportu se zruší.

## 2. Ukončení aplikace a správa záložek (Tab Management)

### Scénář 2.1: Zavírání aplikace s neuloženými záložkami (Cancel)

**Popis:** Ověření, že uživatel může přerušit zavírání aplikace, pokud má neuloženou práci.

- **DÁNO** aplikace je otevřená a obsahuje alespoň jednu neuloženou záložku (indikováno hvězdičkou \* v názvu záložky).
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **PAK** se zobrazí varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Cancel".
- **PAK** se dialog zavře a aplikace zůstane otevřená.

### Scénář 2.2: Uložení neuložených záložek při zavírání aplikace

**Popis:** Ověření, že uživatel může úspěšně uložit svou práci během procesu ukončování aplikace.

- **DÁNO** aplikace je otevřená s neuloženou záložkou.
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **A** zobrazí se varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Save".
- **PAK** Uloží se daný soubor.
- **A** aplikace se ukončí.

### Scénář 2.3: Zahození změn při zavírání aplikace

**Popis:** Ověření, že uživatel může úmyslně zahodit změny a vynutit zavření aplikace.

- **DÁNO** aplikace je otevřená s neuloženou záložkou.
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **A** zobrazí se varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Discard".
- **PAK** jsou neuložené změny ignorovány a aplikace se ukončí.

## 3. Průzkumník souborů a validace odkazů

### Scénář 3.1: Bezpečné zpracování neplatných relativních odkazů

**Popis:** Ověření, že kliknutí na nefunkční odkaz v živém náhledu neshodí aplikaci.

- **DÁNO** uživatel si prohlíží markdown soubor v editoru.
- **A** panel živého náhledu (Live Preview) je viditelný.
- **KDYŽ** uživatel v náhledu klikne na relativní odkaz, který směřuje na neexistující soubor (např. `./chybejici_dokument.md`).
- **PAK** se zobrazí kritický dialog s titulkem "Link Error".

- **A** aplikace zůstane stabilní a aktivní.

### **Scénář 3.2: Bezpečné ošetření smazání souboru mimo aplikaci**

**Popis:** Ověření, že aplikace bezpečně naskakuje i v případě, že byl dříve otevřený soubor smazán operačním systémem.

- **DÁNO** uživatel má v záložce otevřený soubor poznámky .md.
- **A** uživatel ukončí aplikaci.
- **KDYŽ** uživatel smaže soubor poznámky .md pomocí průzkumníku souborů ve Windows/-macOS.
- **A** uživatel znovu spustí aplikaci.
- **PAK** aplikace úspěšně naskakuje bez pádu.
- **A** záložka pro poznámky .md se buď neobnoví, nebo se zobrazí čistá, prázdná pracovní plocha bez otevřené záložky.

## **4. Nastavení a uchování stavu (State Persistence)**

### **Scénář 4.1: Zapamatování rozměrů a rozložení okna**

**Popis:** Ověření, že aplikace obnoví svou přesnou velikost a pozici na obrazovce.

- **DÁNO** aplikace je spuštěna v okně (není maximalizována).
- **KDYŽ** uživatel změní velikost okna na specifický rozměr a přesune ho do rohu obrazovky.
- **A** uživatel aplikaci zavře a znovu spustí.
- **PAK** se aplikace otevře v úplně stejné pozici a velikosti na obrazovce.

### **Scénář 4.2: Obnovení dříve otevřených záložek**

**Popis:** Ověření, že pracovní prostředí uživatele zůstane zachováno přesně tak, jak ho opustil.

- **DÁNO** uživatel má otevřené přesně tři různé markdown soubory ve třech záložkách (A .md, B .md, C .md).
- **A** záložka B .md je aktuálně aktivní/zaměřená.
- **KDYŽ** uživatel aplikaci zavře (a uloží případné změny).
- **A** uživatel aplikaci znovu spustí.
- **PAK** se všechny tři záložky (A .md, B .md, C .md) automaticky obnoví.
- **A** záložka B .md je nastavena jako aktivní/zaměřená.

### **Scénář 4.3: Obnovení dříve otevřeného pracovního adresáře**

**Popis:** Ověření, že otevřený pracovní adresář v panelu Průzkumníka zůstane zachován přesně tak, jak ho opustil.

- **DÁNO** uživatel má otevřený pracovní adresář.
- **KDYŽ** uživatel aplikaci zavře.
- **A** uživatel aplikaci znovu spustí.
- **PAK** zobrazí se daný pracovní adresář v panelu Průzkumníka.



## 5. Živý editor a přepínače rozhraní

### Scénář 5.1: Vykreslování živého náhledu v reálném čase

**Popis:** Ověření, že živý náhled okamžitě reaguje na změny v editoru.

- **DÁNO** uživatel má otevřený markdown soubor.
- **A** zaškrtačací pole "Live Preview" je zaškrtnuté a panel náhledu je viditelný.
- **KDYŽ** uživatel do textového editoru napíše *# Nový nadpis*.
- **PAK** se panel živého náhledu okamžitě aktualizuje a zobrazí zformátovaný HTML nadpis *H1*, aniž by bylo nutné soubor uložit.

### Scénář 5.2: Přepínání panelů uživatelského rozhraní (konkrétně "Analyzer")

**Popis:** Ověření, že UI čistě skrývá a obnovuje panely.

- **DÁNO** uživatel má zobrazený hlavní pohled s markdownem.
- **KDYŽ** uživatel odškrtně pole "Analyzer".
- **PAK** seznam analyzátoru kompletně zmizí z rozložení a uvolní editoru více místa na obrazovce.
- **KDYŽ** uživatel pole "Analyzer" znovu zaškrtně.
- **PAK** se seznam analyzátoru znovu objeví a zachová si svůj předchozí obsah.

### Scénář 5.3: Přepínání panelů uživatelského rozhraní (konkrétně "Live Preview")

**Popis:** Ověření, že UI čistě skrývá a obnovuje panely.

- **DÁNO** uživatel má zobrazený hlavní pohled s markdownem.
- **KDYŽ** uživatel odškrtně pole "Live Preview".
- **PAK** živý náhled kompletně zmizí z rozložení a uvolní editoru více místa na obrazovce.
- **KDYŽ** uživatel pole "Live Preview" znovu zaškrtně.
- **PAK** se živý náhled znovu objeví s aktualizovaným vzhledem, pokud došlo ke změně obsahu v markdown souboru.

## 6. Jádru Markdown analyzátoru

### Scénář 6.1: Označení chybějících povinných nadpisů

**Popis:** Ověření, že analyzátor detekuje porušení struktury šablony.

- **DÁNO** uživatel načetl adresář projektu obsahující šablonu (`template_pattern.md`), která vyžaduje nadpis *## Úkoly*.
- **KDYŽ** uživatel otevře soubor odpovídající kategorii této šablony, ale v souboru chybí nadpis *## Úkoly*.
- **PAK** se v panelu Analyzer objeví specifické varování/chyba upozorňující na chybějící požadovanou strukturu.

### Scénář 6.2: Analyzátor vymaže chyby po jejich opravě

**Popis:** Ověření, že se analyzátor dynamicky aktualizuje, když uživatel chybu opraví.

- **DÁNO** panel Analyzer aktuálně zobrazuje strukturální chybu (např. chybějící povinný nadpis).
- **KDYŽ** uživatel chybějící nadpis do editoru dopíše.
- **A** uživatel klikne na "Save Changes".
- **PAK** se chyba z panelu Analyzer odstraní a zobrazí se stav "Passed" nebo "OK".

## 7. Integrace Průzkumníka souborů (File Explorer)

### Scénář 7.1: Dvojité kliknutí otevře novou záložku

**Popis:** Ověření, že průzkumník správně otevírá nové záložky.

- **DÁNO** uživatel má v panelu Průzkumníka otevřenou složku projektu.
- **A** soubor `sample_pattern.md` NENÍ aktuálně otevřený v žádné záložce.
- **KDYŽ** uživatel dvakrát klikne na `sample_pattern.md` ve stromu Průzkumníka.
- **PAK** se vytvoří nová záložka pro `sample_pattern.md`.
- **A** nová záložka se stane aktivní (zaměřenou).

### Scénář 7.2: Dvojité kliknutí na již otevřený soubor jej zaměří

**Popis:** Ověření, že průzkumník nevytváří duplicitní záložky pro stejný soubor.

- **DÁNO** uživatel má v panelu Průzkumníka otevřenou složku projektu.
- **A** soubor `sample_pattern.md` je již otevřený v Záložce 1 (záložka má stejný název jako soubor).
- **A** uživatel si aktuálně prohlíží Záložku 2.
- **KDYŽ** uživatel dvakrát klikne na `sample_pattern.md` ve stromu Průzkumníka.
- **PAK** aplikace NEVYTVOŘÍ duplicitní záložku.
- **A** aplikace okamžitě přepne zaměření zpět na Záložku 1.

## 8. Automatické verzování na pozadí (Background Auto-Stager)

### Scénář 8.1: Uložení souboru spustí auto-stager

**Popis:** Ověření, že správce Gitu potichu připravuje soubory na pozadí.

- **DÁNO** uživatel má v editoru otevřený sledovaný (tracked) markdown soubor.
- **A** Git repozitář nemá žádné necommitnuté změny.
- **KDYŽ** uživatel provede textovou úpravu a klikne na "Save Changes".
- **PAK** se soubor uloží na disk.
- **A** do konzole v pohledu Git se přidá výstupní zpráva potvrzující, že konkrétní soubor byl úspěšně připraven (auto-staged) na pozadí.

## 9. Horní menu a klávesové zkratky (Top Menu & Shortcuts)

### Scénář 9.1: Otevření souborů a složek přes File menu

**Popis:** Ověření, že položky menu a zkratky pro otevírání vyvolají správné systémové dialogy.

- **DÁNO** aplikace je spuštěna a aktivní.
- **KDYŽ** uživatel vybere v horním menu *File* → *Open File* (nebo stiskne *Ctrl+O*).
- **PAK** se otevře systémový dialog pro výběr .md souboru.
- **KDYŽ** uživatel vybere *File* → *Open Folder* (nebo stiskne *Ctrl+F*).
- **PAK** se otevře systémový dialog pro výběr adresáře projektu.

### Scénář 9.2: Ukládání pomocí klávesové zkratky

**Popis:** Ověření, že klávesová zkratka pro uložení správně ukládá aktivní záložku.

- **DÁNO** uživatel má aktivní záložku s neuloženými změnami (indikováno \*).
- **KDYŽ** uživatel vybere *File* → *Save File* (nebo stiskne *Ctrl+S*).
- **PAK** se aktuální soubor uloží na disk.
- **A** indikátor neuložených změn okamžitě zmizí.

### Scénář 9.3: Přepínání panelů přes File menu (Toggles)

**Popis:** Ověření, že menu správně synchronizuje stav UI panelů.

- **DÁNO** uživatel se nachází v hlavním pohledu (Markdown scene).
- **KDYŽ** uživatel vybere *File* → *Open Explorer* (nebo stiskne *Ctrl+E*).
- **PAK** se boční panel průzkumníka souborů skryje (pokud byl zobrazen) nebo zobrazí (pokud byl skryt).
- **A KDYŽ** uživatel stiskne *Ctrl+P* (Live Preview) nebo *Ctrl+A* (Analyzer).
- **PAK** se příslušné panely v editoru zobrazí nebo skryjí a jejich zaškrtnutí políčka (checkboxy) se automaticky aktualizují.

### Scénář 9.4: Spouštění základních Git operací přes menu

**Popis:** Ověření, že klávesové zkratky spouští procesy na pozadí správně.

- **DÁNO** uživatel má otevřený repozitář.
- **KDYŽ** uživatel vybere *Git* → *Status* (nebo stiskne *Ctrl+Shift+S*).
- **PAK** se operace spustí.
- **A** i když uživatel není aktuálně v pohledu Gitu (Git scene), po přepnutí do něj uvidí výsledek v konzoli.
- (Poznámka: Tento test se opakuje analogicky pro *Fetch* / *Ctrl+Shift+F* a *Pull* / *Ctrl+Shift+L*).

### Scénář 9.5: Ochranné prvky u kritických Git akcí přes menu

**Popis:** Ověření, že vyvolání riskantních akcí přes menu nebo zkratku neobejde naše ochranné prvky (validaci zprávy a varování o ztrátě dat).

- **DÁNO** textové pole pro commit zprávu je prázdné.

- **KDYŽ** uživatel stiskne *Ctrl+Shift+P* (Push).
- **PAK** se operace zastaví a vyskočí chybový dialog "Missing Commit Message".
- **A KDYŽ** uživatel stiskne *Ctrl+Shift+E* (Export Staged).
- **PAK** vyskočí varovný dialog "Warning: Potential Data Loss" úplně stejně, jako by uživatel kliknul na fyzické tlačítko.